Weekly Assessment-7

1.Database Design:

```
CREATE DATABASE student_management;
USE student_management;
```

2. Createing Tables with Constraints:

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age > 15),
  city VARCHAR(50) DEFAULT 'Unknown'
);
```

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	C
1	student_id	int(11)			No	None	
2	name	varchar(50)	utf8mb4_general_ci		No	None	
3	email	varchar(100)	utf8mb4_general_ci		Yes	NULL	
4	age	int(11)			Yes	NULL	
5	city	varchar(50)	utf8mb4_general_ci		Yes	Unknown	
6	phone	varchar(15)	utf8mb4_general_ci		Yes	NULL	

```
CREATE TABLE Courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(50) NOT NULL,
```

```
  credits INT DEFAULT 3
);
```

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default
1	course_id	int(11)			No	None
2	course_name	varchar(50)	utf8mb4_general_ci		No	None
3	credits	int(11)			Yes	3

```
CREATE TABLE Enrollments (
  enroll_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  enroll_date DATE,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default
1	enroll_id	int(11)			No	None
2	student_id	int(11)			Yes	NULL
3	course_id	int(11)			Yes	NULL
4	enroll_date	date			Yes	NULL

```
CREATE TABLE Marks (
  mark_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  marks INT CHECK (marks >= 0),
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default
1	mark_id	int(11)			No	None
2	student_id	int(11)			Yes	NULL
3	course_id	int(11)			Yes	NULL
4	marks	int(11)			Yes	NULL

2.1 Insertion

INSERT INTO Students VALUES

(1,'Amit','amit@gmail.com',20,'Bangalore'),

(2,'Ravi','ravi@gmail.com',21,'Mysore'),

(3,'Sneha','sneha@gmail.com',22,'Delhi'),

(4,'Kiran','kiran@gmail.com',23,'Mumbai'),

(5,'Priya','priya@gmail.com',20,'Chennai'),

(6,'Rahul','rahul@gmail.com',24,'Hyderabad'),

(7,'Anjali','anjali@gmail.com',22,'Pune'),

(8,'Vikram','vikram@gmail.com',23,'Delhi'),

(9,'Neha','neha@gmail.com',21,'Kolkata'),

(10,'Arjun','arjun@gmail.com',25,'Bangalore');

Output:

student_id	name	email	age	city
1	Amit	amit@gmail.com	20	Bangalore
2	Ravi	ravi@gmail.com	21	Bangalore
3	Sneha	sneha@gmail.com	22	Delhi
4	Kiran	kiran@gmail.com	23	Mumbai
5	Priya	priya@gmail.com	20	Chennai
6	Rahul	rahul@gmail.com	24	Hyderabad
7	Anjali	anjali@gmail.com	22	Pune
8	Vikram	vikram@gmail.com	23	Delhi
9	Neha	neha@gmail.com	21	Kolkata
10	Arjun	arjun@gmail.com	25	Bangalore
101	Amit	amit@gmail.com.copy	20	Bangalore
102	Ravi	ravi@gmail.com.copy	21	Bangalore
103	Sneha	sneha@gmail.com.copy	22	Delhi
104	Kiran	kiran@gmail.com.copy	23	Mumbai
105	Priya	priya@gmail.com.copy	20	Chennai
106	Rahul	rahul@gmail.com.copy	24	Hyderabad
107	Anjali	anjali@gmail.com.copy	22	Pune
108	Vikram	vikram@gmail.com.copy	23	Delhi
109	Neha	neha@gmail.com.copy	21	Kolkata
110	Arjun	arjun@gmail.com.copy	25	Bangalore

INSERT INTO Courses VALUES

(101,'DBMS',4),

(102,'Java',3),

(103,'Python',3),

(104,'Web Dev',3),

(105,'AI',4);

Output:

course_id	course_name	credits
101	DBMS	4
102	Java	3
103	Python	3
104	Web Dev	3
105	AI	4

INSERT INTO Enrollments VALUES

(1,1,101,'2026-01-01'),

(2,1,102,'2026-01-02'),

(3,2,103,'2026-01-03'),

(4,3,101,'2026-01-04'),

(5,4,104,'2026-01-05'),

(6,5,105,'2026-01-06'),

(7,6,101,'2026-01-07'),

(8,7,102,'2026-01-08'),

(9,8,103,'2026-01-09'),

(10,9,104,'2026-01-10'),

(11,10,105,'2026-01-11'),

(12,2,101,'2026-01-12'),

(13,3,102,'2026-01-13'),

(14,4,103,'2026-01-14'),

(15,5,104,'2026-01-15');

Output:

enroll_id	student_id	course_id	enroll_date
1	1	101	2026-01-01
2	1	102	2026-01-02
3	2	103	2026-01-03
4	3	101	2026-01-04
5	4	104	2026-01-05
6	5	105	2026-01-06
7	6	101	2026-01-07
8	7	102	2026-01-08
9	8	103	2026-01-09
10	9	104	2026-01-10
11	10	105	2026-01-11
12	2	101	2026-01-12
13	3	102	2026-01-13
14	4	103	2026-01-14
15	5	104	2026-01-15

INSERT INTO Marks VALUES

(1,1,101,85),(2,1,102,78),

(3,2,103,88),(4,3,101,90),

(5,4,104,70),(6,5,105,95),

(7,6,101,60),(8,7,102,75),

(9,8,103,82),(10,9,104,77),

(11,10,105,91),(12,2,101,65),

(13,3,102,89),(14,4,103,73),

(15,5,104,80),(16,6,101,55),

(17,7,102,68),(18,8,103,92),

(19,9,104,74),(20,10,105,88);

Output:

mark_id	student_id	course_id	marks
1	1	101	85
2	1	102	78
3	2	103	88
4	3	101	90
5	4	104	70
6	5	105	95
7	6	101	60
8	7	102	75
9	8	103	82
10	9	104	77
11	10	105	91
12	2	101	65
13	3	102	89
14	4	103	73
15	5	104	80
16	6	101	55
17	7	102	68
18	8	103	92
19	9	104	74
20	10	105	88

3. SQL Queries

3.1 Basic Queries

SELECT * FROM Students;

Output: you can see in 2.1

SELECT name, city FROM Students WHERE city='Bangalore';

Output:

name	city
Amit	Bangalore
Ravi	Bangalore
Arjun	Bangalore
Amit	Bangalore
Ravi	Bangalore
Arjun	Bangalore

SELECT * FROM Students ORDER BY age DESC;

Output:

student_id	name	email	age 1	city
110	Arjun	arjun@gmail.com.copy	25	Bangalore
10	Arjun	arjun@gmail.com	25	Bangalore
106	Rahul	rahul@gmail.com.copy	24	Hyderabad
6	Rahul	rahul@gmail.com	24	Hyderabad
108	Vikram	vikram@gmail.com.copy	23	Delhi
4	Kiran	kiran@gmail.com	23	Mumbai
8	Vikram	vikram@gmail.com	23	Delhi
104	Kiran	kiran@gmail.com.copy	23	Mumbai
7	Anjali	anjali@gmail.com	22	Pune
3	Sneha	sneha@gmail.com	22	Delhi
107	Anjali	anjali@gmail.com.copy	22	Pune
103	Sneha	sneha@gmail.com.copy	22	Delhi
102	Ravi	ravi@gmail.com.copy	21	Bangalore
109	Neha	neha@gmail.com.copy	21	Kolkata
2	Ravi	ravi@gmail.com	21	Bangalore
9	Neha	neha@gmail.com	21	Kolkata
105	Priya	priya@gmail.com.copy	20	Chennai
101	Amit	amit@gmail.com.copy	20	Bangalore
5	Priya	priya@gmail.com	20	Chennai
1	Amit	amit@gmail.com	20	Bangalore

SELECT * FROM Students LIMIT 5;

Output:

student_id	name	email	age 1	city
10	Arjun	arjun@gmail.com	25	Bangalore
110	Arjun	arjun@gmail.com.copy	25	Bangalore
106	Rahul	rahul@gmail.com.copy	24	Hyderabad
6	Rahul	rahul@gmail.com	24	Hyderabad
104	Kiran	kiran@gmail.com.copy	23	Mumbai

3.2 Logical Operators

SELECT * FROM Students WHERE age > 20 AND city='Delhi';

student_id	name	email	age	city
3	[illegible]	[illegible]	22	Delhi
8	[illegible]	[illegible]	23	Delhi
103	Sneha	sneha@gmail.com.copy	22	Delhi
108	Vikram	vikram@gmail.com.copy	23	Delhi

- Drag to reorder.
- Click to mark/unmark.
- Double-click to copy column name.

SELECT * FROM Students WHERE city='Bangalore' OR city='Mysore';

student_id	name	email	age 1	city
10	Arjun	arjun@gmail.com	25	Bangalore
110	Arjun	arjun@gmail.com.copy	25	Bangalore
2	Ravi	ravi@gmail.com	21	Bangalore
102	Ravi	ravi@gmail.com.copy	21	Bangalore
1	Amit	amit@gmail.com	20	Bangalore
101	Amit	amit@gmail.com.copy	20	Bangalore

SELECT * FROM Students WHERE NOT city='Delhi';

student_id	name	email	age 1	city
ick the drop-down arrow		arjun@gmail.com.copy	25	Bangalore
toggle column's visibility.		arjun@gmail.com	25	Bangalore
106	Rahul	rahul@gmail.com.copy	24	Hyderabad
6	Rahul	rahul@gmail.com	24	Hyderabad
4	Kiran	kiran@gmail.com	23	Mumbai
104	Kiran	kiran@gmail.com.copy	23	Mumbai
7	Anjali	anjali@gmail.com	22	Pune
107	Anjali	anjali@gmail.com.copy	22	Pune
109	Neha	neha@gmail.com.copy	21	Kolkata
2	Ravi	ravi@gmail.com	21	Bangalore
102	Ravi	ravi@gmail.com.copy	21	Bangalore
9	Neha	neha@gmail.com	21	Kolkata
105	Priya	priya@gmail.com.copy	20	Chennai
101	Amit	amit@gmail.com.copy	20	Bangalore
5	Priya	priya@gmail.com	20	Chennai
1	Amit	amit@gmail.com	20	Bangalore

3.3 Aggregate Functions

SELECT MIN(marks) FROM Marks;

```
SELECT MIN(marks) FROM Marks;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

MIN(marks)
55

SELECT MAX(marks) FROM Marks;

```
SELECT MAX(marks) FROM Marks;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

MAX(marks)
95

SELECT COUNT(*) FROM Students;

```
SELECT COUNT(*) FROM Students;
```

Profiling [Edit inline] [Edit] [Explain SQ

Extra options

COUNT(*)
20

SELECT AVG(marks) FROM Marks;

```
SELECT AVG(marks) FROM Marks;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

AVG(marks)
78.7500

SELECT SUM(marks) FROM Marks;

```
SELECT SUM(marks) FROM Marks;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

SUM(marks)
1575

3.4 Filtering

SELECT * FROM Students WHERE name LIKE 'A%';

student_id	name	email	age	city
1	Amit	amit@gmail.com	20	Bangalore
7	Anjali	anjali@gmail.com	22	Pune
10	Arjun	arjun@gmail.com	25	Bangalore
101	Amit	amit@gmail.com.copy	20	Bangalore
107	Anjali	anjali@gmail.com.copy	22	Pune
110	Arjun	arjun@gmail.com.copy	25	Bangalore

SELECT * FROM Students WHERE city IN ('Delhi','Mumbai');

student_id	name	email	age 1	city
4	Kiran	kiran@gmail.com	23	Mumbai
8	Vikram	vikram@gmail.com	23	Delhi
104	Kiran	kiran@gmail.com.copy	23	Mumbai
108	Vikram	vikram@gmail.com.copy	23	Delhi
3	Sneha	sneha@gmail.com	22	Delhi
103	Sneha	sneha@gmail.com.copy	22	Delhi

SELECT * FROM Students WHERE age BETWEEN 20 AND 23;

student_id	name	email	age 1	city
104	Kiran	kiran@gmail.com.copy	23	Mumbai
8	Vikram	vikram@gmail.com	23	Delhi
108	Vikram	vikram@gmail.com.copy	23	Delhi
4	Kiran	kiran@gmail.com	23	Mumbai
3	Sneha	sneha@gmail.com	22	Delhi
7	Anjali	anjali@gmail.com	22	Pune
107	Anjali	anjali@gmail.com.copy	22	Pune
103	Sneha	sneha@gmail.com.copy	22	Delhi
2	Ravi	ravi@gmail.com	21	Bangalore
9	Neha	neha@gmail.com	21	Kolkata
102	Ravi	ravi@gmail.com.copy	21	Bangalore
109	Neha	neha@gmail.com.copy	21	Kolkata
105	Priya	priya@gmail.com.copy	20	Chennai
101	Amit	amit@gmail.com.copy	20	Bangalore
5	Priya	priya@gmail.com	20	Chennai
1	Amit	amit@gmail.com	20	Bangalore

3.5 Joins

3.5.1 INNER JOIN

SELECT s.name, c.course_name

FROM Students s

INNER JOIN Enrollments e ON s.student_id=e.student_id

INNER JOIN Courses c ON e.course_id=c.course_id;

name	course_name
Amit	DBMS
Sneha	DBMS
Rahul	DBMS
Ravi	DBMS
Amit	Java
Anjali	Java
Sneha	Java
Ravi	Python
Vikram	Python
Kiran	Python
Kiran	Web Dev
Neha	Web Dev
Priya	Web Dev
Priya	AI
Arjun	AI

3.5.2 LEFT JOIN

SELECT s.name, c.course_name

FROM Students s

LEFT JOIN Enrollments e ON s.student_id=e.student_id

LEFT JOIN Courses c ON e.course_id=c.course_id;

name	course_name
Amit	DBMS
Amit	Java
Ravi	Python
Ravi	DBMS
Sneha	DBMS
Sneha	Java
Kiran	Web Dev
Kiran	Python
Priya	AI
Priya	Web Dev
Rahul	DBMS
Anjali	Java
Vikram	Python
Neha	Web Dev
Arjun	AI
Amit	NULL
Ravi	NULL
Sneha	NULL
Kiran	NULL
Priya	NULL
Rahul	NULL
Anjali	NULL
Vikram	NULL
Neha	NULL
Arjun	NULL

3.5.3 RIGHT JOIN

SELECT s.name, c.course_name

FROM Students s

RIGHT JOIN Enrollments e ON s.student_id=e.student_id

RIGHT JOIN Courses c ON e.course_id=c.course_id;

name	course_name
Amit	DBMS
Sneha	DBMS
Rahul	DBMS
Ravi	DBMS
Amit	Java
Anjali	Java
Sneha	Java
Ravi	Python
Vikram	Python
Kiran	Python
Kiran	Web Dev
Neha	Web Dev
Priya	Web Dev
Priya	AI
Arjun	AI

3.5.4 CROSS JOIN

SELECT s.name, c.course_name

FROM Students s CROSS JOIN Courses c;

name	course_name
Amit	DBMS
Amit	Java
Amit	Python
Amit	Web Dev
Amit	AI
Ravi	DBMS
Ravi	Java
Ravi	Python
Ravi	Web Dev
Ravi	AI
Sneha	DBMS
Sneha	Java
Sneha	Python
Sneha	Web Dev
Sneha	AI
Kiran	DBMS
Kiran	Java
Kiran	Python
Kiran	Web Dev
Kiran	AI
Priya	DBMS
Priya	Java
Priya	Python
Priya	Web Dev
Priya	AI

3.5.5 SELF JOIN

SELECT A.name, B.name

FROM Students A, Students B

WHERE A.city = B.city AND A.student_id <> B.student_id;

name	name
Ravi	Amit
Arjun	Amit
Amit	Amit
Ravi	Amit
Arjun	Amit
Amit	Ravi
Arjun	Ravi
Amit	Ravi
Ravi	Ravi
Arjun	Ravi
Vikram	Sneha
Sneha	Sneha
Vikram	Sneha
Kiran	Kiran
Priya	Priya
Rahul	Rahul
Anjali	Anjali
Sneha	Vikram
Sneha	Vikram
Vikram	Vikram
Neha	Neha
Amit	Arjun
Ravi	Arjun
Amit	Arjun
Ravi	Arjun

3.6 Grouping

SELECT course_id, AVG(marks)

FROM Marks

GROUP BY course_id

HAVING AVG(marks) > 75;

course_id	AVG(marks)
102	77.5000
103	83.7500
104	75.2500
105	91.3333

4. Advanced

4.1 UNION

SELECT name FROM Students WHERE city='Delhi'

UNION

SELECT name FROM Students WHERE city='Mumbai';

name
Sneha
Vikram
Kiran

4.2 EXISTS

```
SELECT name FROM Students s

WHERE EXISTS (

   SELECT 1 FROM Marks m WHERE m.student_id=s.student_id

);
```

name
Amit
Ravi
Sneha
Kiran
Priya
Rahul
Anjali
Vikram
Neha
Arjun

4.3 CASE

```
SELECT student_id, marks,

CASE

   WHEN marks > 85 THEN 'Excellent'

   WHEN marks >= 70 THEN 'Good'

   ELSE 'Average'

END AS grade

FROM Marks;
```

student_id	marks	grade
1	85	Good
1	78	Good
2	88	Excellent
3	90	Excellent
4	70	Good
5	95	Excellent
6	60	Average
7	75	Good
8	82	Good
9	77	Good
10	91	Excellent
2	65	Average
3	89	Excellent
4	73	Good
5	80	Good
6	55	Average
7	68	Average
8	92	Excellent
9	74	Good
10	88	Excellent

5. Data Modification

5.1 UPDATE

UPDATE Students SET city='Bangalore' WHERE student_id=2;

5.2 DELETE

DELETE FROM Students WHERE age < 20;

5.3 INSERT SELECT

INSERT INTO Students (student_id,name,email,age,city)

SELECT student_id+100, name, email, age, city FROM Students;

0 rows affected. (Query took 0.0003 seconds.)

```
UPDATE Students SET city='Bangalore' WHERE student_id=2;
```

[Edit inline] [Edit] [Create PHP code]

0 rows deleted. (Query took 0.0003 seconds.)

```
DELETE FROM Students WHERE age < 20;
```

[Edit inline] [Edit] [Create PHP code]

20 rows inserted. (Query took 0.0025 seconds.)

```
INSERT INTO Students (student_id, name, email, age, city) SELECT student_id + 200, name, CONCAT(email, '.new2'), age, city FROM Students;
```

[Edit inline] [Edit] [Create PHP code]

6. Advanced Features

6.1 NULL Handling

SELECT name, IFNULL(city,'No City') FROM Students;

name	IFNULL(city,'No City')
Amit	Bangalore
Ravi	Bangalore
Sneha	Delhi
Kiran	Mumbai
Priya	Chennai
Rahul	Hyderabad
Anjali	Pune
Vikram	Delhi
Neha	Kolkata
Arjun	Bangalore
Amit	Bangalore
Ravi	Bangalore
Sneha	Delhi
Kiran	Mumbai
Priya	Chennai
Rahul	Hyderabad
Anjali	Pune
Vikram	Delhi
Neha	Kolkata
Arjun	Bangalore
Amit	Bangalore
Ravi	Bangalore
Sneha	Delhi
Kiran	Mumbai
Priya	Chennai

SELECT COALESCE(city,'Unknown') FROM Students;

COALESCE(city,'Unknown')
Bangalore
Bangalore
Delhi
Mumbai
Chennai
Hyderabad
Pune
Delhi
Kolkata
Bangalore
Bangalore
Bangalore
Delhi
Mumbai
Chennai
Hyderabad
Pune
Delhi
Kolkata
Bangalore
Bangalore
Bangalore
Delhi
Mumbai
Chennai

6.2 View

```
CREATE VIEW top_students AS

SELECT student_id, marks

FROM Marks

WHERE marks > 85;
```

student_id	marks
2	88
3	90
5	95
10	91
3	89
8	92
10	88

6.3 Stored Procedure

```
DELIMITER //

CREATE PROCEDURE GetStudentMarks(IN sid INT)

BEGIN

  SELECT * FROM Marks WHERE student_id = sid;

END //

DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0046 seconds.)

```
CREATE PROCEDURE GetStudentMarks(IN sid INT) BEGIN SELECT * FROM Marks WHERE student_id = sid; END;
```

Edit inline] [Edit] [Create PHP code]

```
CALL GetStudentMarks(1);
```

mark_id	student_id	course_id	marks
1	1	101	85
2	1	102	78

6.4 Index & Alter

CREATE INDEX idx_name ON Students(name);

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0243 seconds.)

```
CREATE INDEX idx_name ON Students(name);
```

[Edit inline] [Edit] [Create PHP code]

ALTER TABLE Students ADD phone VARCHAR(15);

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0182 seconds.)

```
ALTER TABLE Students ADD phone VARCHAR(15);
```

Edit inline] [Edit] [Create PHP code]